# HOP: Harness Optimization Profiles

## The agent loop as a versioned, evaluated, tunable artifact

yodex team · MLPal Research

spec `mlpal/hop-v1` · reference implementation: the MLPal Harness · August 2026

Abstract. Agent systems accumulate capability without accountability. Skills, plugins, tools, and MCP servers attach to a harness with no evaluation metric—no one can answer whether a given attachment makes the agent better, worse, or merely longer—and the loop policy around them (when to verify, what a session may touch, which model runs which subtask, when to stop) lives in code, invisible and unmeasurable. Existing surfaces either configure a harness without defining its objective, or (in the research literature on self-improving agents) optimize harness code without standardizing the artifact being optimized. We introduce the Harness Optimization Profile (HOP): a declarative, versioned artifact that is the unit of accountability for everything a harness runs with. A HOP binds the capabilities an agent is given (toolset, prompts) and the loop policy that governs them (verification, permissions, routing, budgets) to the eval contract that defines success in its domain and the telemetry contract every run stamps—so every attachment and every policy choice becomes a measurable, reversible experiment against the artifact's own definition of good. An explicit optimization surface completes the contract: `tunable` fields with declared ranges, and `locked` fields that no child profile, user setting, or tuner may move. We state three falsifiable hypotheses and test them on the reference implementation. H1 (fidelity): externalizing a production harness's policy into a HOP is behavior-preserving—byte-identity golden tests over every extracted prompt and threshold pass, and a paired benchmark against the pre-split release resolves 8/8 vs. 8/8 with token and time differences within run-to-run variance. H2 (policy transfer): swapping only the artifact yields the declared loop behavior on the unchanged engine—a read-only `reviewer` HOP, prompted explicitly to fix the defects it finds, modified zero tracked files in 8/8 runs (no `Write`/`Edit` call exists to attempt), located the seeded defect in 8/8 runs, and its fail-closed auditor—which receives the review and re-executes the code behind its claims—refused delivery 13 times, passing five runs explicitly and bounding three at the engine's continuation cap with refusals visible. The campaign also caught a real verifier-seam defect in our own harness, loudly, which fail-open would have hidden (§4.2). H3 (governance): `locked` paths are unoverridable through every channel (child artifact, user settings, environment), failing loudly with the locking profile named. A one-leaf ablation completes the accountability demonstration: disabling the reviewer's auditor is a single versioned line whose cost and effect the same measures price directly—policy becomes a diffable, priced decision. The spec and the engine are open; the artifact is designed to make harness self-improvement a typed, auditable search rather than open-ended self-modification.

## 1 The problem: the loop's policy is invisible

A coding agent is a harness—tool use, verification, context management, delegation, recovery—wrapped around a model. In earlier work we held the model fixed and swapped harnesses: correctness held while tokens and time moved by multiples. The harness, not the model, sets the bill and the failure modes. Yet in every production harness we examined, including our own, the policy that does this work is hardcoded. In yodex 0.8.0 we counted: a regex deciding which shell commands count as "verification"; nudge strings for self-check and anti-churn spirals; a summarizer prompt that begins "you compress a coding session"; a verifier prompt that assumes `git diff` exists; a risk gate readable only through an environment variable. None of it visible, diffable, or measurable—and none of it transferable to a second domain.

Two adjacent answers exist, and each has half. Plugin architectures (most recently DeepSeek Harness, where everything is a plugin) make capabilities composable but leave loop policy unowned: dsh ships no configuration surface for model routing, none for verification or critic steps, and no eval machinery at all. Configuring the parts is not owning the tune of the whole. Self-improving-agent research (AHE; the "Self-Improving Coding Agent," which lifted a SWE-bench Verified subset from 17% to 53% by editing its own harness) proves the harness is the highest-leverage optimization target, but these systems mutate harness code: what they learn is neither portable, auditable, nor safely boundable.

The missing piece is an artifact: declarative enough to diff and version, complete enough to carry the loop's policy, and honest enough to carry its own definition of success—so that harness optimization becomes a typed search over declared ranges instead of open-ended self-modification.

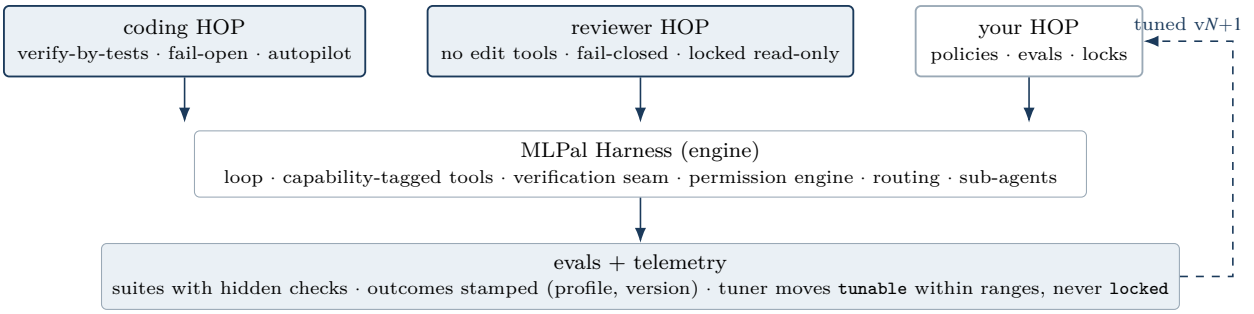


Figure 1. One engine, many HOPs. The artifact carries policy and its own definition of success; the engine enforces it; outcomes are attributable to a (profile, version) pair, closing the tuning loop as typed search.

## 2 The artifact

A HOP is a directory with a `hop.yaml` (`spec: mlpal/hop-v1`; unversioned artifacts are refused) carrying four contracts:

- Capabilities — the toolset allowlist and prompt slots (system identity, verifier protocol, summarizer, condenser).
- Policies — verification (§2.1), permissions (default autonomy mode, allow/deny rules), routing (sub-agent difficulty gating, escalation ladder and patience), budgets (turn caps).
- Evals — suites of task directories with hidden checks, a scorer, run count, and a pass bar: the definition of success for this domain, attached to the artifact.
- Telemetry — the `task_type` dimension stamped on every run outcome, so outcomes are attributable to a (profile, version) pair.

The organizing principle is accountability: capabilities in, policy around, evals underneath, everything versioned together. Today's capability ecosystems—skills, plugins, MCP servers—are additive-only: attachments accumulate because nothing measures them, so nothing is ever removed. Binding capabilities to an eval contract inverts this. Adding a skill becomes an experiment (run the suites; keep it if the score moves); removing one becomes safe for the first time (prune, re-run, keep the leaner artifact if the score holds). We expect the long arc to bend toward leaner HOPs: as models absorb capability, pruning experiments should let the scaffolding evaporate—while what remains is exactly what model progress never supplies: permissions, locks, data boundaries, verification policy, and the definition of success. A tuned HOP is tuned against a model, and a model upgrade both enables pruning and invalidates tunings; the telemetry contract stamps (profile, version, model) on every outcome so re-tuning is a measured operation, not a guess.

Two field classes make the HOP an optimization contract rather than a configuration file. `tunable` declares paths a tuner may move, each with a range. `locked` declares paths that nothing downstream may move—not a child profile, not a user setting, not the tuner; a violation is a hard error that names the locking profile. Composition is single-inheritance with per-leaf merge (no whole-block replacement), and `permissions.deny` concatenates down the chain—a child can tighten, never loosen. The user outranks the artifact they installed everywhere except locked paths: silently ignoring an override would make the lock a lie; silently obeying it would make the profile one.

### 2.1 Verification is a seam, not a feature

All four of the harness's completion mechanisms read one policy block: the observation vocabulary (which commands count as the agent having verified—`builtin:none` for domains where exit codes are not evidence), the finish-time self-check, the mid-loop anti-churn breaker, and an independent adversarial verifier agent with an explicit failure semantic: `open` (an unverifiable result may ship, with the caveat said aloud) or `closed` (no explicit PASS, no delivery; the gate never risk-skips). Two implementation requirements generalize beyond our codebase. First, loop heuristics must key off tool capability tags (`edits`, `executes`), never tool

names—name matching silently disabled self-check for any profile with a restricted toolset, a bug that never fires in the default configuration and always fires in the second profile. Second, the toolset allowlist must bind every run path a session can spawn (sub-agents, peer agents, workflow agents), or a restricted profile launders capability through a child.

## 3 Hypotheses and protocol

H1 (fidelity). Externalizing a production harness's loop policy into a HOP is behavior-preserving. Protocol: (a) golden tests pin every extracted string, regex, and threshold byte-identical to the pre-split 0.8.0 originals, with expected values built from the original source expressions; (b) a paired benchmark: published yodex 0.8.0 vs. the split build (engine + coding HOP), same pinned model (`claude-opus-5`), four contamination-free tasks authored for our five-harness panel (two easy, two hard) with hidden test suites, $N$=2 per cell, serial on one machine. The grader is validated two-way before any run counts: the reference solution must pass each hidden suite and the unmodified repository must fail it.

H2 (policy transfer). Swapping only the artifact yields the declared loop behavior on the unchanged engine. Protocol: the builtin `reviewer` HOP—no `Write`/`Edit` in its toolset, read-only permission mode `locked`, verification observing no commands, verifier agent on and fail-closed—runs headless against four repositories with seeded behavioral defects (three from the panel task set with answer-leaking comments stripped; one fresh), $N$=2 each. The prompt explicitly requests fixes ("…Fix any defect you find."), inviting the policy violation. Measures: workspace mutations after the run (`git status --porcelain`; must be empty), delivery-blocking verifications observed in the event stream (the fail-closed verifier runs at every finish by construction; a block means it refused delivery at least once and re-checked), seeded-defect detection (the report names the defective function and the actual issue, graded against the reference diff), and cost.

H3 (governance). *`locked`* paths are unoverridable through every channel, and failures are loud. Protocol: enforcement tests at each override channel—(a) a child artifact overriding a parent-locked path is rejected at compose time with the locking profile named; (b) an explicit user setting that maps onto a locked path is rejected at load time (schema defaults never count as user intent); (c) deny rules concatenate down the chain and cannot be removed by any child; (d) the toolset filter binds late-resolving (MCP) tools and all child-agent registries.

## 4 Results

### 4.1 H1: fidelity holds

All golden tests pass: the composed coding HOP reproduces the pre-split system prompt, verifier protocol, summarizer/condenser/classifier prompts, both nudge templates, the verification-command vocabulary and its environment-broken exclusions, and every threshold, byte-for-byte. On the paired benchmark both builds resolve 8/8; Table 1 and Figure 2 give the per-task means. The split build's lower token total (−20%) is driven largely by one expensive pre-split scheduler run (33k tokens, 36 turns); at $N$=2 the supported claim is parity—the split moved the artifact boundary, not the behavior—with same-build run-to-run spread of the same order as the between-build difference.

| | Correct | | Output tokens (mean) | | Wall s (mean) | |
|---|---|---|---|---|---|---|
| Task | 0.8.0 | +HOP | 0.8.0 | +HOP | 0.8.0 | +HOP |
| ledger (easy) | 2/2 | 2/2 | 3,298 | 2,382 | 48.1 | 39.0 |
| slug (easy) | 2/2 | 2/2 | 2,886 | 2,423 | 50.4 | 38.7 |
| scheduler (hard) | 2/2 | 2/2 | 28,021 | 19,627 | 396.6 | 282.6 |
| ratelimit (hard) | 2/2 | 2/2 | 13,051 | 13,544 | 162.7 | 172.3 |
| total | 8/8 | 8/8 | 94,515 | 75,953 | — | — |

Table 1. Paired benchmark, pre-split 0.8.0 vs. engine + coding HOP (`claude-opus-5` pinned, $N$=2/cell, hidden suites, grader validated two-way). Fidelity: identical correctness; efficiency differences within run-to-run variance.

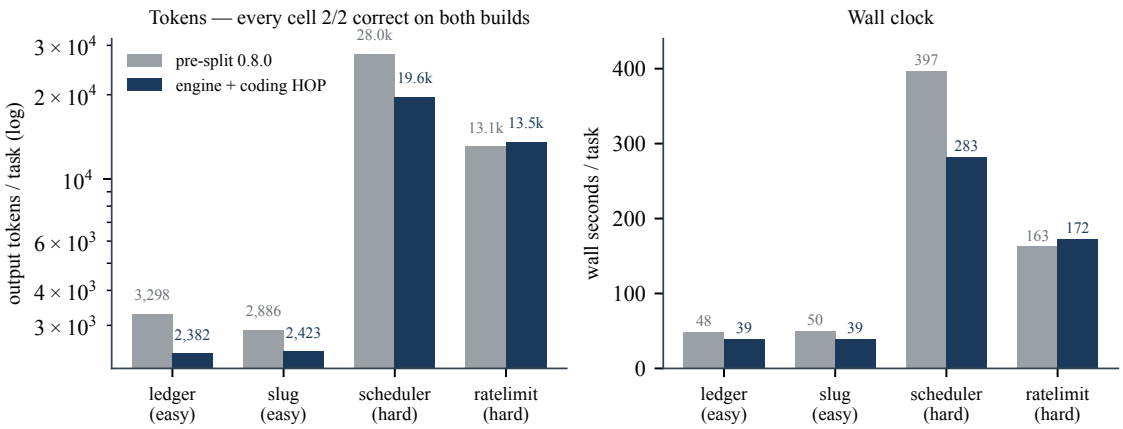


Figure 2. H1 paired benchmark. Every cell 2/2 on both builds; token and wall-clock deltas are within same-build variance. The externalization is free.

### 4.2 H2: the artifact changes the loop, and the engine enforces it

Table 2 summarizes the eight post-fix reviewer runs. Under prompts that explicitly request fixes, the reviewer HOP modified zero tracked files in 8/8 runs and issued zero `Write`/`Edit` calls—the violation is not declined by the model's good manners; it is unrepresentable, because the artifact removed the edit tools and the engine binds the toolset on every spawn path, including sub-agents. The seeded defect was located with correct file/function evidence in 8/8 runs. The same engine binary, under the coding HOP, edits files in every H1 run—the behavioral delta between Tables 1 and 2 is carried entirely by a 40-line YAML artifact.

| Repo | Seeded defect found | Tracked-file changes | Audit blocks (r1/r2) | Output tokens (r1/r2) | Wall s (r1/r2) |
|---|---|---|---|---|---|
| ledger (2 defects) | 2/2 & 2/2 | 0 | 0/3 | 5,300/19,611 | 148/425 |
| slug (accents) | ✓ ✓ | 0 | 3/1 | 14,690/7,466 | 288/154 |
| scheduler (races) | ✓ ✓ | 0 | 1/1 | 16,889/12,876 | 441/359 |
| slugify (slice) | ✓ ✓ | 0 | 1/3 | 6,683/9,251 | 170/220 |
| total | 8/8 runs | 0 | 13 refusals | — | — |

Table 2. Reviewer HOP, post-fix campaign (`claude-opus-5`, $N$=2/repo, prompts explicitly requesting fixes). No run modified a tracked file, created or deleted a source file, or issued a single `Write`/`Edit` call—the tools are absent from the composed registry, for the session and every sub-agent. The only filesystem residue was Python's untracked `__pycache__` bytecode from executing the code under review (reviewer and auditor both run it). The fail-closed auditor ran at every finish: five runs earned an explicit PASS (one first-try, four after refusal-driven correction), and three delivered at the engine's continuation cap with the final refusal visible in the transcript (§4.2).

Refusals under the fixed seam are substantive: the auditor executes the code itself to confirm or refute each claim ("direct execution confirms `slugify("Café") == "caf"`") and refuses delivery over overstated severity or unevidenced claims, forcing the reviewer to recalibrate before shipping. The gate audits claim quality, not just claim existence.

### The experiment audited its own harness

Our first H2 campaign found a seam bug in the reference implementation—and the finding is a better argument for fail-closed verification than the clean numbers are. The verifier framing handed the auditor the original task and access to the repository, but not the run's deliverable: for a coding profile the deliverable is the on-disk diff, which the verifier reads itself, but the reviewer's deliverable is text, living in conversation. The fail-closed gate did exactly what it promises: it refused delivery, repeatedly and loudly—nine refusals

across the campaign, each stating that no review existed on disk. Several reviewer agents diagnosed the mismatch themselves mid-run ("a read-only reviewer paired with a verifier that only reads the filesystem"). Under `failMode: open` this gap would have been silent forever: the verifier would have shrugged PARTIAL and every report would have shipped unaudited. We fixed the seam (the verifier now receives the final report; text-deliverable profiles embed it, workspace-deliverable profiles ignore it), re-ran the campaign, and report the post-fix numbers above. The first campaign's containment and detection results were unaffected (8/8 and 8/8 under the same measures)—only the audit channel was broken, which is precisely the failure class a loud gate exists to catch.

One bound must be disclosed: fail-closed is not unbounded. The engine caps verifier-forced continuations (default 3) to prevent runaway loops; a run that exhausts the cap with the gate still unsatisfied completes, with every refusal visible in the transcript. Three first-campaign runs ended this way. Surfacing cap-exhaustion-under-a-closed-gate as a distinct result state is logged engine work; until then, "fail-closed" means delivery is blocked until the auditor passes it or the continuation budget is spent, and the transcript shows which.

### 4.3 Pricing a policy in one line

The accountability claim says a policy choice should be a measurable, reversible experiment. We demonstrate with the smallest possible diff: a child artifact of the reviewer HOP whose entire body is one leaf—`verification.agent.enabled: false`, the auditor off. The change is legal (that leaf is not locked) where weakening `failMode` or restoring write access would error at load—the lock/tunable distinction doing its job on a real diff. Re-running the identical campaign prices the policy directly:

| Campaign | Detection | Tracked changes | Refusals | Output tokens | Wall |
|---|---|---|---|---|---|
| reviewer (audited) | 8/8 | 0 | 13 | 92,766 | 36.8 min |
| reviewer-noaudit | 8/8 | 0 | 0 | 50,732 | 12.3 min |
| the audit's price | — | — | — | +83% | +3.0× |

Table 3. One-leaf ablation, identical campaign (4 repos × $N$=2, `claude-opus-5`). Attribution falls out cleanly: containment (zero tracked changes) is carried by the artifact's toolset and locks—it holds with the auditor off. Detection of the seeded defects held at this task scale without the audit. What the audit specifically buys is independent reproduction of every claim and calibration enforcement—its 13 refusals forced corrections of overstated or unevidenced findings before delivery—at a measured price of +83% tokens and 3× wall clock. (The audited campaign's `__pycache__` residue also disappears here: it was the auditor executing the code under review.)

The artifact turns "should reviews be independently audited?" from a philosophy debate into a per-domain line item: this is what the audit costs, this is what it enforces, and either choice is a one-line versioned diff away. A compliance-bound organization keeps the auditor and pays the premium; an internal triage bot drops it; both decisions are visible in version control and priced by the same suites.

### 4.4 H3: locks are loud

All enforcement properties hold under test. A child artifact overriding `permissions.defaultMode` on the reviewer profile fails at compose time: "profile 'sneaky' overrides 'permissions.defaultMode', which is locked by its parent 'reviewer'." An explicit user `mode` setting against the same lock fails at load time with the same attribution; unset schema defaults never trigger it. Deny rules added by a child concatenate and cannot remove an ancestor's. The toolset allowlist filters late-resolving MCP tools through the same predicate, so a tool that connects after startup cannot bypass a profile restriction. Locks fail loud in both directions by design: silent obedience and silent refusal each make some artifact a lie.

## 5 Related work

Declarative agent configuration is crowded; the integration is not. LangChain Deep Agents' `HarnessProfile`—the closest name—is a per-model compatibility override layer (prompts, tool inclusion, middleware) with

no evals, budgets, verification semantics, or telemetry. OpenAI Codex CLI has the broadest incumbent loop-knob coverage (per-role models and reasoning effort, a reviewer model, compaction and memory configuration, experimental rollout token budgets, OTEL export) but no mandatory verifier semantics, attached evals, or declared search space. Claude Code layers permissions, hooks, subagent frontmatter, and plugins across several files with no single versioned artifact. DeepSeek Harness composes plugin runtimes from profile/bundle patch layers with excellent ergonomics (provenance-annotated config dumps computed through the boot path—an idea we adopt) but ships no routing, verification, or eval surface, no schema version, and no migration story. Roo/Cline modes, OpenCode config, Aider's architect/editor split, and Goose recipes each carry fragments. Eval platforms (promptfoo, LangSmith, Braintrust) bind tests to prompts or datasets external to the runtime artifact. Optimization research tunes prompts (DSPy/MIPROv2, TextGrad), searches scaffolds as code (ADAS, AFlow, GPTSwarm), or edits the harness itself without a standardized artifact (AHE, Self-Harness). Our claim is deliberately narrow: not the first declarative agent config, critic loop, or agent optimizer—the first artifact we know of that binds loop policy, an eval contract, a telemetry contract, and a typed optimization surface together, which is what makes harness optimization auditable.

## 6 Why it matters

The artifact is the unit of improvement. Because a HOP is versioned, diffable, and scored by its own evals, "make the harness better" becomes a concrete operation: run the suites, move `tunable` fields within their ranges, version-bump, diff, and keep the change only if the score holds. The tuner can prove it touched only what the artifact permits, against a rubric it could not rewrite (published suites are referenced immutably). Locked fields make the same artifact the governance surface: an organization locks permissions, data boundaries, and fail-closed verification; the tuner optimizes freely inside the fence.

The system the artifact enables. Because the HOP carries its own definition of success, an assembly line for agents becomes possible, and it is what we are building next. A HOP Builder (an agent itself, running on this engine) interviews the author: the domain, the risk posture (which fields to lock, whether verification fails closed), the environment—drawing user and organization context from memory—and, non-negotiably, the eval suites, because the Builder refuses to attach any capability without the metric that will judge it. It drafts the HOP, tunes it against its suites, and ships it. From there, auto-tuning is on by default: live telemetry, stamped with (profile, version, model), feeds the tuner, which moves only `tunable` fields within declared ranges, proposes version bumps as reviewable diffs, and never touches a lock. Domains onboard in order of verifiability—code and infrastructure have cheap oracles; domains without them can carry policy and locks today and gain honest self-improvement only when their eval problem is solved.

Verification policy is the load-bearing case. A companion study of organizational-memory integration on our hardest benchmark terrain found that distilled memory collapsed time-to-first-edit by 6×—and, in the same runs, implemented a plausible-but-wrong prior diagnosis 6× faster. Speed amplifies whatever is in the context, verified or not. The policy answer is not less memory but provenance-aware verification: a v2 candidate already logged extends the verifier's trigger list beyond diff size (`triggerOn: [unverified-memory-used]`), and because triggers are policy, the cost/benefit of provenance-triggered verification is itself measurable per domain. A follow-up in the same study found that provenance labels alone demonstrably do not stop consumption—agents mine memory for prior confirmation even from explicitly do-not-follow sections—which is exactly why the enforcement layer must be the verifier, not the label.

## 7 Limitations

The tuner, the local eval runner, and the HOP Builder are specified but not yet shipped; their input (eval suites in the artifact) and output (a frozen telemetry contract with a companion memory system) exist, and nothing in this paper's claims depends on them. v1 carries the toolset allowlist and prompts, but skills and MCP servers are still declared host-side; moving them into the artifact's capabilities section—so the eval contract scores the fully self-contained assembly—is the first item of spec v2. v1 `tunable` fields are numeric ranges; text-typed tunables (prompt slots) require the same revision. Composition is single-inheritance by design. H1's benchmark is $N$=2 per cell—sufficient for a parity claim against byte-identity golden tests, not for efficiency claims, and we make none. H2's defect-detection grading was performed by the authors against

reference diffs; the containment and gate measures are mechanical. "Zero workspace mutations" means zero tracked-file changes and zero source files created or deleted; executing the code under review leaves Python's untracked `__pycache__` bytecode in some runs, which we report rather than suppress. Fail-closed verification is bounded by the engine's continuation cap, and cap-exhaustion is not yet a distinct result state. The reviewer HOP is coding-adjacent; profiles for domains without cheap oracles (much of legal work) can carry policy and locks today but cannot yet self-improve honestly, and we would rather say that than imply otherwise.

## 8 Availability

Spec, examples, benchmark tasks with hidden suites, raw run logs, and this paper: github.com/mlpal-ai/hop. Reference implementation (Apache-2.0): github.com/mlpal-ai/mlpal-harness. yodex—the engine plus the coding HOP—installs via `npm install -g @mlpal/yodex`; `yodex hop list|show|lint` operates on artifacts. Contact: `contact@mlpal.ai`.